

Why jQuery?

With jQuery, not only do you get a very high level of cross-browser compatibility, but you also have quick and easy access to HTML and DOM manipulation, special functions to interact directly with CSS, the ability to control events, powerful tools to create professional effects and animations, and functions for conducting asynchronous communications with the web server. jQuery is also the base for a wide range of plugins and other utilities.

Of course, you don't *need* to use jQuery, and some programming purists would never touch a library, preferring to create their own bespoke collections of functions (and there can be good reasons for this, such as not having to wait on others to correct bugs you find, implementing your own security features, and so on). But jQuery has definitely stood the test of time, and if you would like to take advantage of its gentle learning curve and be developing quality web pages as quickly as possible, this chapter will show how you can get started using it.



Important as jQuery is, being used on perhaps 90% of production websites, there are new kids on the block that have been drawing a lot of praise and attention. And because technology changes fast, as well as understanding jQuery, you need to keep up with newly surfacing technologies that could supersede the ones you are currently using. In my opinion React, Angular, and Vue are a few of these technologies as they offer new, interesting, and powerfully scalable ways to enhance JavaScript (as you would expect, having been created by Facebook, Google, and ex-Google staff, respectively). Although we cannot fill this book with JavaScript frameworks, it is important to keep abreast of the main ones, so I have chosen to also introduce you to React in [Chapter 24](#), as I believe it is the framework you are likely to come into contact with most frequently, although you may wish to check out Angular, too, for completeness.

Including jQuery

There are two ways to include jQuery in your web pages. You can go to the [download page](#), download the version you need, upload it to your web server, and reference it from a `<script>` tag in your HTML files. Or you can take advantage of a free content delivery network (CDN) and simply link to the version you require.



jQuery is released under the terms of the MIT license, which places almost no restrictions on what you can do with it. You are free to use any jQuery project in any other project (even commercial projects) as long as the copyright header is left intact.

Choosing the Right Version

Before deciding whether to download and host jQuery directly or to use a CDN, you must pick a version of jQuery. In most cases this is straightforward, because you'll simply opt for the latest release. However, if you are targeting particular browsers, or if you are maintaining a legacy website that relies on a particular version of jQuery, the latest one may not be right for you.

Unlike most software, where you simply download and install the newest available version, jQuery has evolved over time to take account of the changing dynamics in the market of differing browser versions, with different features and bugs.

At the same time, improvements to jQuery have been made that might make newer versions operate differently on sites that have been specially tailored to a particular version (and the quirks surrounding it).

Of course, each newer version is an improvement over the previous one and is more and more likely to cover all the bases. But where identical operation is critical for a website, until you have fully tested a new version, it is often best to stick with an earlier one.

Different flavors of jQuery

There are now three branches of jQuery, called 1.x, 2.x, and 3.x, each designed for different environments.

Version 1.x was the first stable release of jQuery. This release supports older web browsers that aren't even supported by their respective developers any longer. If you expect a large number of visitors with older web browsers, this is the version to go for (as I write, version 1.12 is probably the best).

Version 2.x dropped support for Internet Explorer 6–8 to increase jQuery's overall performance and reduce the library's file size. It is faster and smaller than version 1.x but doesn't support older web browsers. Since Microsoft dropped support for Windows XP, it may be safe to assume that your visitors will be using a browser compatible with version 2.x, unless you know otherwise.

If you need to support older browsers like Internet Explorer 6–8, Opera 12.1x, or Safari 5.1+, the jQuery developers recommend using version 1.12. For full details on the different versions supported, please refer to the [website](#). In this edition of the book, I have settled on version 3.5.1.

Compressed or editable

You also must decide whether to use a version of jQuery that has been minified (compressed in size to minimize bandwidth and decrease loading time) or an uncompressed version (perhaps because you want to make an edit to it yourself—which you

are fully entitled to do). Generally, a minified version is best, but most web servers support *gzip* for on-the-fly compression and decompression, so this is becoming less important (although minification does remove comments too).

Downloading

The most recent released version of jQuery is available in both uncompressed and minified forms on the [download page](#). You can also find all of the past releases on the [jQuery CDN](#). The slim versions of jQuery that appear on the download page omit the asynchronous communication functions to save on space, so you should avoid these if you wish to make use of any Ajax functionality using jQuery.

All you need to do is choose the version you need, right-click the link displayed alongside it, and save it to your hard disk. From there, you can upload it to your web server and then include it within `<script>` tags, like this (for the minified version of release 3.5.1):

```
<script src='http://myserver.com/jquery-3.5.1.min.js'></script>
```



If you have never used jQuery before (and have no special requirements), then just download the latest minified version, or link to it via a CDN, as described in the following section.

Using a Content Delivery Network

Several CDNs support jQuery. If you use one of them, you can save yourself the hassle of having to download new versions and then upload them to your server by simply linking directly to the URLs supplied by these networks.

Not only that, but they provide this service free of charge, and usually on high-capacity backbones that are probably the fastest in the world. Additionally, CDNs usually hold their content in a number of different geographic locations and then supply the required file from the server closest to a user, ensuring the quickest possible delivery.

Overall, if you don't need to modify the jQuery source code (requiring you to host it on your own web servers), and your users are certain to have a live internet connection, using a CDN is probably the way to go. And it's very easy. All you need to know is the filename you wish to access and the root folder the CDN is using. For example, all current and previous versions can be accessed through the CDN that jQuery uses, like this:

```
<script src='http://code.jquery.com/jquery-3.5.1.min.js'></script>
```

The base directory is at <http://code.jquery.com/>, and you simply follow this with the name of the file you need to include (in this case, *jquery-3.5.1.min.js*).

Both Microsoft and Google offer jQuery on their networks, so you can also include it in either of the following ways:

```
<script src='http://ajax.aspnetcdn.com/ajax/jquery/jquery-3.5.1.min.js'></script>  
<script src='http://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js'>  
</script>
```

In the case of the **Microsoft CDN**, you should begin the URL with the base directory of *ajax.aspnetcdn.com/ajax/jquery/* and then follow that with the filename you require.

For Google, however, you must split the filename (for example, *jquery-3.5.1.min.js*) into a folder and filename (like this: *3.5.1/jquery.min.js*). Then precede that with *ajax.googleapis.com/ajax/libs/jquery/*.



An added benefit of using a CDN is that most other websites also do this, so jQuery may well already be cached in the user's browser and might not even require redelivery. With 90% or more of websites using jQuery, this can be a lot of valuable bandwidth and time saved.

Customizing jQuery

If it's absolutely critical that you keep the amount of data downloaded by a web page to the minimum, then you may still be able to use jQuery by making a special build of it that includes only the features your website will be using. You won't be able to rely on a CDN to deliver it, but in this circumstance you probably weren't planning on using one anyway.

To create your own custom build of jQuery, try **jQuery Builder**. Simply check the boxes you want and uncheck those that you don't. The bespoke version of jQuery will then be loaded into a separate tab or window, from where you can copy and paste it as required.

jQuery Syntax

The most striking thing about jQuery to people who are new to it is the `$` symbol, which acts as the jQuery factory method—the main means of access into the framework. It was chosen because the symbol is legal in JavaScript, is short, and is different from customary variable, object, or function/method names.

This symbol takes the place of making a call to the jQuery function (which you can also do if you wish). The idea is to keep your code short and sweet and to save on

unnecessary extra typing each time you access jQuery. It also immediately shows other developers new to your code that jQuery (or a similar library) is in use.

A Simple Example

At its simplest, you access jQuery by typing a `$` symbol, followed by a selector in parentheses, and then a period and a method to apply to the selected element(s).

For example, to change the font family of all paragraphs to monospace, you could use this statement:

```
$('p').css('font-family', 'monospace')
```

Or to add a border to a `<code>` element, you could use this:

```
$('code').css('border', '1px solid #aaa')
```

Let's look at that as part of a complete example (see [Example 22-1](#), where the jQuery parts are highlighted in bold).

Example 22-1. A simple jQuery example

```
<!DOCTYPE html>
<html>
  <head>
    <title>First jQuery Example</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    The jQuery library uses either the <code>$()</code>
    or <code>jQuery()</code> function names.
    <script>
      $(<b>'code'</b>).css(<b>'border'</b>, '<b>1px solid #aaa</b>')
    </script>
  </body>
</html>
```

When you load this example into a browser, the result will be similar to [Figure 22-1](#). Of course, this particular instruction simply replicates what you can do with normal CSS, but the idea is to illustrate jQuery syntax, so I'm keeping things simple for now.



Another way of issuing this command is by calling the jQuery function (which works in the same way as `$`), like this:

```
jQuery('code').css('border', '1px solid #aaa')
```

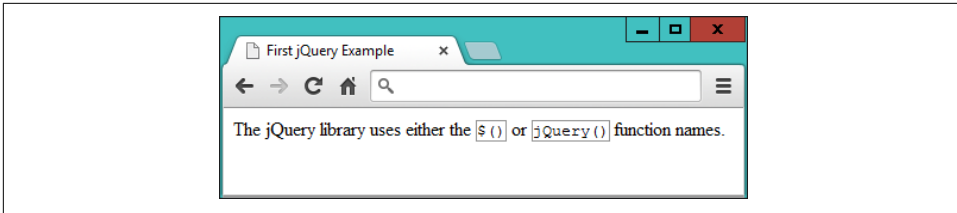


Figure 22-1. Modifying elements with jQuery

Avoiding Library Conflicts

If you use other libraries alongside jQuery, you may find that they define their own `$` functions. To resolve this issue, you can call the `noConflict` method on the symbol, which releases control so that the other library can take over, like this:

```
$.noConflict()
```

Once you do this, to access jQuery thereafter, you must call the `jQuery` function. Or you can replace the `$` symbol with an object name of your choice, like this:

```
jq = $.noConflict()
```

Now you can use the keyword `jq` to call jQuery wherever you had previously used `$`.



To distinguish and keep track of jQuery objects separately from standard element objects, some developers prefix a `$` to any object created with jQuery (so that they end up looking like PHP variables!).

Selectors

Now that you've seen how easy it is to include jQuery in a web page and access its features, let's move on to looking at its selectors, which (I'm sure you'll be pleased to learn) work in exactly the same way as in CSS. In fact, selectors are at the heart of how most of jQuery operates.

All you have to do is think about how you would style one or more elements using CSS, and then you can use the same selector(s) to apply jQuery operations on these selected elements. This means you can make use of element selectors, ID selectors, class selectors, and any combinations thereof.

The `css` Method

To explain jQuery's use of selectors, let's first look at one of the more fundamental jQuery methods, `css`, with which you can dynamically alter any CSS property. It takes two arguments, the property name to be accessed and a value to be applied, like this:

```
css('font-family', 'Arial')
```

As you will see in the following sections, you cannot use this method on its own; you must append it to a jQuery selector, which will select one or more elements whose properties should be changed by the method. The following, which sets the content of all `<p>` elements to display with full justification, is an example:

```
$('#p').css('text-align', 'justify')
```

You can also use the `css` method to return (rather than set) a computed value by supplying only a property name (and no second argument). In this case, the value of the first element that matches the selector is returned. For example, the following will return the text color of the element with the ID of `elem` as an `rgb` method:

```
color = $('#elem').css('color')
```

Remember that the value returned is the *computed* value. In other words, jQuery will compute and return the value as used by the browser at the moment the method is called, not the original value that may have been assigned to the property via a stylesheet or in any other way.

So, if the text color is blue (for example), the value assigned to the variable `color` in the preceding statement will be `rgb(0, 0, 255)`, even if the color was originally set with the color name `blue`, or the hex strings `#00f` or `#0000ff`. This computed value, though, will always be in a form that can be assigned back to the element (or any other element) via the second argument of the `css` method.



Be wary of any computed dimensions returned by this method because, depending on the current `box-sizing` setting (see [Chapter 20](#)), they may or may not necessarily be what you expect. When you need to get or set widths and heights without consideration for `box-sizing`, you should use the `width` and `height` methods (and their siblings), as described in the section “[Modifying Dimensions](#)” on page 547.

The Element Selector

To select an element to be manipulated by jQuery, just list its name within the parentheses following the `$` symbol (or jQuery function name). For example, if you wanted to change the background color of all `<blockquote>` elements, you could use a statement such as the following:

```
$('#blockquote').css('background', 'lime')
```

The ID Selector

You can also refer to elements by their IDs if you place a # character in front of the ID name. So, to add a border to the element with the ID of advert (for example), you could use this:

```
$('#advert').css('border', '3px dashed red')
```

The Class Selector

And you can manipulate groups of elements according to the class they use. For example, to underline all elements that use the class new, you could use this:

```
$('.new').css('text-decoration', 'underline')
```

Combining Selectors

Just as with CSS, you may combine selectors into a single jQuery selection using commas, as with the following example:

```
$('blockquote, #advert, .new').css('font-weight', 'bold')
```

Example 22-2 brings all these types of selectors together into a single example (with the jQuery statements shown in bold), the result of which you can see in **Figure 22-2**.

Example 22-2. Using jQuery with different selectors

```
<!DOCTYPE html>
<html>
  <head>
    <title>Second jQuery Example</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <blockquote>Powerful and flexible as JavaScript is, with a plethora of
      built-in functions, it is still necessary to use additional code for
      simple things that cannot be achieved natively or with CSS, such as
      animations, event handling, and asynchronous communication.</blockquote>
    <div id='advert'>This is an ad</div>
    <p>This is my <span class='new'>new</span> website</p>
    <script>
      $('blockquote').css('background', 'lime')
      $('#advert').css('border', '3px dashed red')
      $('.new').css('text-decoration', 'underline')
      $('blockquote, #advert, .new').css('font-weight', 'bold')
    </script>
  </body>
</html>
```

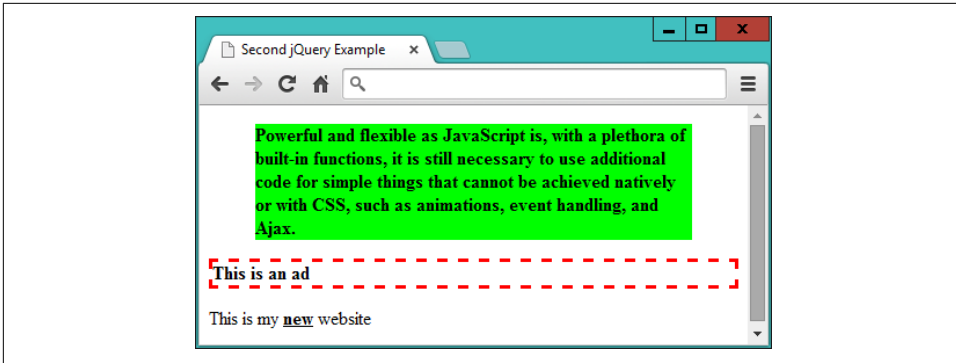


Figure 22-2. Manipulating multiple elements

Handling Events

If all jQuery could do was alter CSS styles, it wouldn't be a great deal of help—but of course, it can do far more than that. Let's further investigate by seeing how it handles events.

As you will recall, most events are triggered by user interaction: when a mouse passes over an element, the mouse button is clicked or a key is pressed. But there are also other events that can be triggered, such as when a document completes loading.

With jQuery, it's a simple matter to attach your own code to these events in a safe way that doesn't block other code from also gaining access to them. For example, here's how to get jQuery to respond to an element being clicked:

```
$('#clickme').click(function()
{
    $('#result').html('You clicked the button!')
})
```

When the element with the ID of `clickme` is clicked, the `innerHTML` property of the element with the ID of `result` is updated via the jQuery `html` function.



jQuery objects (created with either the `$` or `jQuery` methods) are *not* the same as JavaScript objects created with `getElementById`. In plain JavaScript, you can use a statement like `object = document.getElementById('result')` followed by (for example) `object.innerHTML = 'something'`. But in the preceding example, `$('#result').innerHTML` would not work, because `innerHTML` is not a property of a jQuery object—hence the use of the jQuery method `html` to achieve the required result.

Example 22-3 fleshes out the idea; you can see it running in **Figure 22-3**.

Example 22-3. Processing an event

```
<!DOCTYPE html>
<html>
  <head>
    <title>jQuery Events</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <button id='clickme'>Click Me</button>
    <p id='result'>I am a paragraph</p>
    <script>
      $('#clickme').click(function()
      {
        $('#result').html('You clicked the button!')
      })
    </script>
  </body>
</html>
```

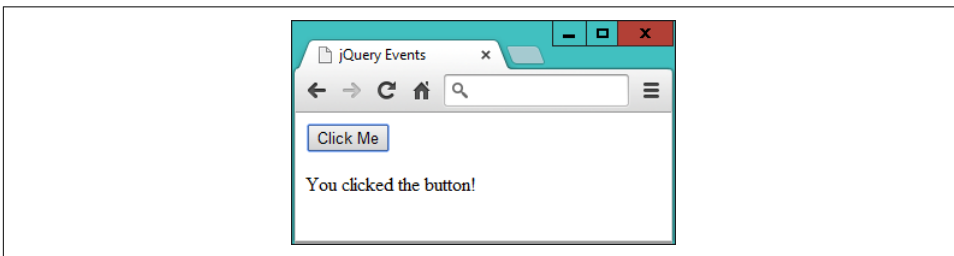


Figure 22-3. Processing a click event



When accessing an event with jQuery, omit the `on` prefix that you would use in standard JavaScript. So, for example, the `onmouseover` event name becomes the `mouseover` function in jQuery, `onclick` becomes `click`, and so on.

Waiting Until the Document Is Ready

Since jQuery is so closely related to the DOM in what it lets you achieve, more often than not you will need to wait until a web page has loaded before manipulating parts of it. Without jQuery, this can be achieved with the `onload` event, but there's a more efficient, cross-browser jQuery method called `ready` that you can call to enable it at the earliest possible moment (even sooner than with `onload`). This means jQuery can get working on a page that much more quickly, and user-unfriendly delays are minimized.

To make use of this feature, place your jQuery code within the following structure:

```
$('#document').ready(function()  
{  
    // Your code goes here  
})
```

The code will wait there until the document is ready, and only then will it be called by the `ready` method. In fact, there's a shorter version you can use that takes even less typing, as shown in [Example 22-4](#).

Example 22-4. The smallest jQuery “ready” code encapsulation function

```
$(function()  
{  
    // Your code goes here  
})
```

If you get used to encapsulating your jQuery statements in one of these two structures, you won't encounter the types of errors that can be generated by trying to access the DOM too soon.



Alternatively, another approach is to always place your JavaScript at the *end* of every HTML page so that it is executed only after the entire document has loaded. There is a secondary advantage too, in that this ensures the web page contents get priority with loading—thus, you may well see improvements in user experience.

The only time end-of-page scripts may not be a good idea is if a document could appear to be ready when it isn't, or if all external stylesheets have not yet loaded (which can really be identified only by testing), causing users to think they can interact with it before your script is ready. In such cases, implement the `ready` function and all will be well. In fact, if in doubt, place your script at the page end *and* place its jQuery calls within the `ready` function, and you'll get the best of both worlds.

Event Functions and Properties

You've just seen the `ready` event method, but you can access several dozen jQuery event methods and associated properties (far too many to detail here). However, the following are some of the ones that are more commonly used, and they'll get you started for most projects. For a comprehensive summary of all available events, please see the [documentation](#).

The blur and focus Events

The blur event triggers when focus is removed from an element, causing it to blur, and is a good partner for the focus event. The blur and focus methods can be used to add a handler to the event. They will trigger the event if you omit any arguments from the method's parentheses.

In **Example 22-5**, there are four input fields. The first is given immediate focus with a quick call to the focus method, applying it to the element with the ID of first. Then a pair of handlers is added to all input elements. The focus handler sets their background to yellow when focus is given, and the blur handler sets their background to light gray when focus is removed (or blurred).

Example 22-5. Using the focus and blur events

```
<!DOCTYPE html>
<html>
  <head>
    <title>Events: blur</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <h2>Click in and out of these fields</h2>
    <input id='first'> <input> <input> <input>
    <script>
      $('#first').focus()
      $('input').focus(function() { $(this).css('background', '#ff0') } )
      $('input').blur(function() { $(this).css('background', '#aaa') } )
    </script>
  </body>
</html>
```



You are allowed to include whitespace characters between the closing parenthesis of a method and the period operator used to attach another method to it (and after the period too, if you like), as in the previous example where I have right-aligned the blur event name under focus, to help the rest of the statements also line up in columns.

In **Figure 22-4**, you can see how this code gives any input fields that have ever had focus a light-gray background color. If one currently has focus, its background color is set to yellow, while unvisited fields remain with a white background color.

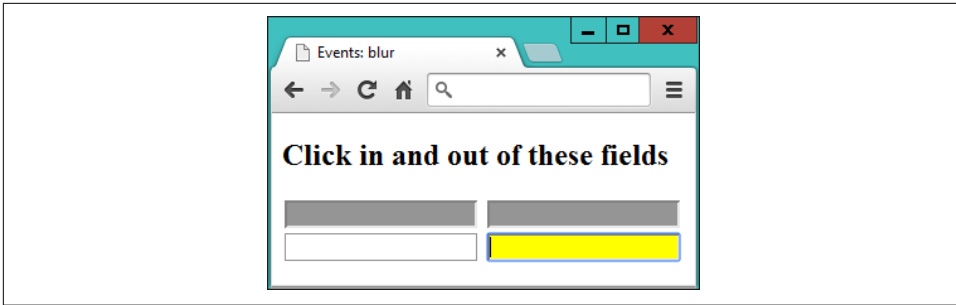


Figure 22-4. Attaching event handlers to the *blur* and *focus* events

The `this` Keyword

This example also serves to illustrate the use of the `this` keyword. When an event is called, the element on which it was triggered is passed in the object `this`, which can then be given to the `$` method for processing. Or, since `this` is a standard JavaScript object (and not a jQuery object), it can be used as such. So, if you prefer, you could replace this:

```
$(this).css('background', '#ff0')
```

with this:

```
this.style.background = '#ff0'
```

The `click` and `dblclick` Events

You saw the `click` event a little earlier, but there's an event for handling double-clicks too. To use either, attach the event's method to a jQuery selection, and for its argument provide a jQuery method to invoke when the event triggers, like this:

```
$('.myclass').click( function() { $(this).slideUp() })
$('.myclass').dblclick( function() { $(this).hide() })
```

Here I have opted to use inline anonymous functions, but you can use named ones instead if you like (just remember to supply only the name of the function, without parentheses, or it will be called at the wrong time). The `this` object will pass through as expected and be made available to the named function, like this:

```
$('.myclass').click(doslide)

function doslide()
{
    $(this).slideUp()
}
```

The `slideUp` and `hide` methods are detailed in the section “[Special Effects](#)” on [page 533](#). For now, though, just try running [Example 22-6](#) and either clicking or

double-clicking the buttons to see how they sometimes disappear with an animation (using `slideUp`) and sometimes just vanish (using `hide`), as shown in [Figure 22-5](#).

Example 22-6. Attaching to the `click` and `dblclick` events

```
<!DOCTYPE html>
<html>
  <head>
    <title>Events: click & dblclick</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <h2>Click and double click the buttons</h2>
    <button class='myclass'>Button 1</button>
    <button class='myclass'>Button 2</button>
    <button class='myclass'>Button 3</button>
    <button class='myclass'>Button 4</button>
    <button class='myclass'>Button 5</button>
    <script>
      $('myclass').click( function() { $(this).slideUp() })
      $('myclass').dblclick( function() { $(this).hide() })
    </script>
  </body>
</html>
```



Figure 22-5. Button 3 has been clicked once and is sliding up

The keypress Event

From time to time, you'll need better control over user keyboard interaction, particularly when processing complex forms or when writing games. For cases such as these, you can use the `keypress` method, which can be attached to anything that accepts keyboard input, such as an input field or even the document itself.

In [Example 22-7](#), the method has been attached to the document in order to intercept all key presses. The result of running it can be seen in [Figure 22-6](#).

Example 22-7. Intercepting key presses

```
<!DOCTYPE html>
<html>
  <head>
    <title>Events: keypress</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <h2>Press some keys</h2>
    <div id='result'></div>
    <script>
      $(document).keypress(function(event)
      {
        key = String.fromCharCode(event.which)

        if (key >= 'a' && key <= 'z' ||
            key >= 'A' && key <= 'Z' ||
            key >= '0' && key <= '9')
        {
          $('#result').html('You pressed: ' + key)
          event.preventDefault()
        }
      })
    </script>
  </body>
</html>
```

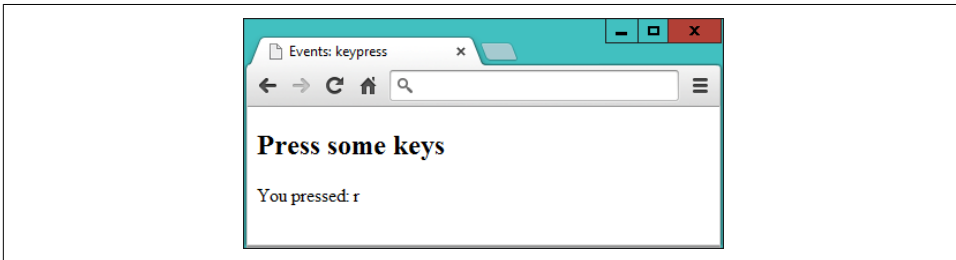


Figure 22-6. Processing key presses from the keyboard

There are a few things of note in this example that you will need to bear in mind when writing your own keyboard handlers. For example, because browsers return differing values for this event, the `which` property of the event object is normalized by jQuery to return the same character codes across all browsers. So, this is where to look for which key was pressed.

Being a character code, though, the value in `which` is a number. You can turn it into a single-letter string by passing it through `String.fromCharCode`. You don't have to do this, because you can easily respond to ASCII values in your code, but this method is useful when you need to work with characters.

These values are taken from the jQuery `pageX` and `pageY` properties, which refer to the mouse pointer's offset from the top-left-hand corner of the containing document. Therefore, since the canvas is itself offset slightly from that location, the canvas offset values (in `offsetLeft` and `offsetTop`) are subtracted from `pageX` and `pageY`:

```
var xpos = event.pageX - canvas.offsetLeft
var ypos = event.pageY - canvas.offsetTop
```

Now that we know where the mouse pointer is in relation to the canvas, the next pair of lines tests the value of `pendown`. If it is `true`, we know that the mouse button is being pressed, so a `lineTo` call is made to draw a line to the current location. Otherwise, the pen is up, and so `moveTo` is called to simply update the current location:

```
if (pendown) context.lineTo(xpos, ypos)
else          context.moveTo(xpos, ypos)
```

Then the `stroke` method is called to apply whichever drawing command was just made to the canvas. These five lines are all that is needed to handle drawing, but it's still necessary to track the mouse button state, so the final two lines of code intercept the `mousedown` and `mouseup` events, setting `pendown` to `true` when the mouse button is pressed and `false` when it's released:

```
$('#pad').mousedown(function() { pendown = true } )
$('#pad') .mouseup(function() { pendown = false } )
```

In this example, you see the combination of three different event handlers working together to create a simple utility, using both local variables for internal expressions and global variables where an object or the state of something must be made available across multiple functions.

Other Mouse Events

The `mouseenter` and `mouseleave` events trigger whenever the mouse passes into an element or leaves it. No position values are supplied because the functions assume you simply want to make a Boolean decision about what to do when one of these events is triggered.

In [Example 22-9](#), a pair of anonymous functions is attached to these events, altering the HTML of an element accordingly, as shown in [Figure 22-8](#).

Example 22-9. Detecting the mouse entering and leaving an element

```
<!DOCTYPE html>
<html>
  <head>
    <title>Events: Further Mouse Handling</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
```

```

<body>
  <h2 id='test'>Pass the mouse over me</h2>
  <script>
    $('#test').mouseenter(function() { $(this).html('Hey, stop tickling!') } )
    $('#test').mouseleave(function() { $(this).html('Where did you go?') } )
  </script>
</body>
</html>

```

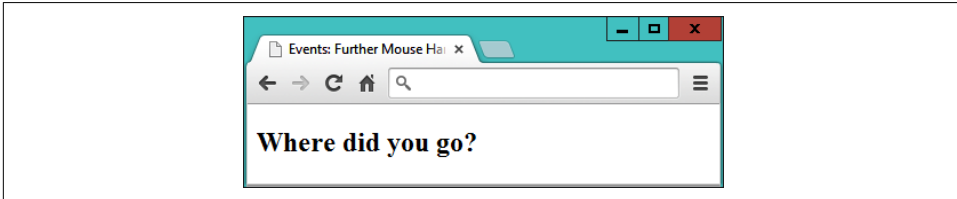


Figure 22-8. Detecting when the mouse enters and leaves an element

When the mouse enters the bounds of the selected element, that element’s `innerHTML` property is updated (with a call to `html`). Then, when the mouse leaves again, a further update is made to the element’s HTML.

Alternative Mouse Methods

A number of other jQuery mouse event functions are available to cover a wide range of circumstances, all of which are detailed in the [mouse events documentation](#). For example, you can use the following `mouseover` and `mouseout` methods to achieve similar results to the code in the preceding section:

```

$('#test').mouseover(function() { $(this).html('Cut it out!') } )
$('#test') .mouseout(function() { $(this).html('Try it this time...') } )

```

Or you could use the `hover` method to bind two handlers with a single function call, like this:

```

$('#test').hover(function() { $(this).html('Cut it out!') },
  function() { $(this).html('Try it this time...') } )

```

If you are planning on creating `mouseover` and `mouseout` combined effects, clearly the `hover` method is the logical function to choose—but there’s also another way you can achieve the same result, which is *chaining* (explained in the section “[Method chaining](#)” on page 541), using code like this:

```

$('#test').mouseover(function() { $(this).html('Cut it out!') } )
  .mouseout(function() { $(this).html('Try it this time...') } )

```

Here the period operator at the start of the second statement attaches it to the first, creating a chain of methods.



The preceding examples show how to capture mouse click, mouse movement, and keyboard events, and therefore they are mostly suitable for desktop environments—which is what jQuery is primarily targeted at. However, there is a version of jQuery for mobile devices that provides all the touch-handling event control you could wish for (and much more), called **jQuery Mobile**. We'll take a closer look at this in the next chapter.

The submit Event

When a form is submitted, you may want to perform some error checking on the data entered before it gets sent to the server. One way to do this is to intercept the submit event of the form, as in **Example 22-10**. **Figure 22-9** shows the result of loading this document and then submitting the form with one or more fields left empty.

Example 22-10. Intercepting the submit event of a form

```
<!DOCTYPE html>
<html>
  <head>
    <title>Events: submit</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <form id='form'>
      First name: <input id='fname' type='text' name='fname'><br>
      Last name:  <input id='lname' type='text' name='lname'><br>
      <input type='submit'>
    </form>
    <script>
      $('#form').submit(function()
      {
        if ($('#fname').val() == '' ||
            $('#lname').val() == '')
        {
          alert('Please enter both names')
          return false
        }
      })
    </script>
  </body>
</html>
```

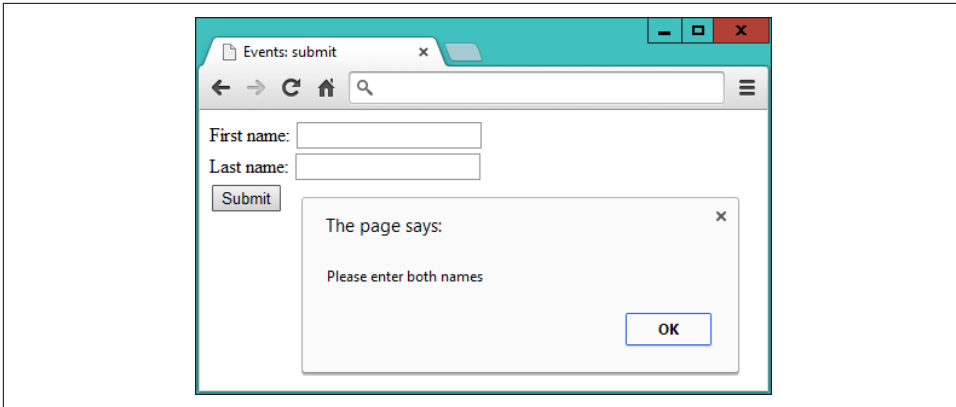


Figure 22-9. Checking user input upon submission

The key parts of this example are where the event is attached to an anonymous function, like this:

```
$('#form').submit(function()
```

and where the values of the two input fields are tested for being empty:

```
if ($('#fname').val() == '' ||  
    $('#lname').val() == '')
```

Here the jQuery `val` method is used to retrieve the value in the `value` property of each field. This is neater than using `$('#fname')[0]` (as in [Example 22-8](#)) to get access to the DOM object and then appending `value` to it to read the field's value, like this: `$('#fname')[0].value`.

In this example, by returning the value `false` if one or more fields are empty, the `if` test cancels the normal submission process. To enable the submission to proceed, you can return `true`, or just not return anything.

Special Effects

jQuery really begins to come into its own when processing special effects. Although you can use CSS3 transitions, they are not always easy to manage dynamically from JavaScript—but with jQuery it really is as simple as selecting one or more elements and then applying one or more effects to them.

The core effects available are hiding and showing, fading in and out, sliding, and animations. These can be used singly, together in synchronization, or in sequence. They also support the use of callbacks, which are functions or methods that get called only once an operation has completed.

The following sections describe some of the more useful jQuery effects, each of which supports up to three arguments, as follows:

Duration

When a duration value is supplied, the effect will take place over the assigned time, which can be a value in milliseconds or the strings `fast` or `slow`.

Easing

There are only two easing options in the jQuery library, `swing` and `linear`. The default is `swing`, which gives a more natural effect than `linear`. For more easing options, you can check out plug-ins like [jQuery UI](#).

Callback

If you supply a callback function, it will be called after the effect method completes.

This means that when no arguments are given, the method is called immediately without being placed in the animation queue.

So, for example, you can call the `hide` method in a variety of ways, such as these:

```
$('#object').hide()
$('#object').hide(1000)
$('#object').hide('fast')
$('#object').hide('linear')
$('#object').hide('slow', 'linear')
$('#object').hide(myfunction)
$('#object').hide(333, myfunction)
$('#object').hide(200, 'linear', function() { alert('Finished!') } )
```

As you'll see in the section "[Method chaining](#)" on page 541, you can attach function calls (with arguments) to each other and they will then be animated in turn, like in the following example, which will hide and then reveal an element:

```
$('#object').hide(1000).show(1000)
```

Other less commonly used arguments are also supported by many of these methods, and you can get full details on them (and all the other supported effects methods) by visiting the [documentation on effects](#).

Hiding and Showing

Probably the simplest effect is to hide and show elements in response to user interaction. As described in the previous section, you can provide no arguments or a variety of arguments to the `hide` and `show` methods. By default, when none are supplied, the result is to instantly hide or reveal an element.

When arguments are supplied, these two methods modify the width, height, and opacity properties of an element simultaneously, until they reach 0 for hide or their

original values for show. The hide function sets the display property of the element to none when it is fully hidden, and the show function reassigns its previous value to it once the element is fully restored.

Example 22-11 lets you try hide and show for yourself (as shown in **Figure 22-10**).

Example 22-11. Hiding and showing an element

```
<!DOCTYPE html>
<html>
  <head>
    <title>Effects: hide & show</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <button id='hide'>Hide</button>
    <button id='show'>Show</button>
    <p id='text'>Click the Hide and Show buttons</p>
    <script>
      $('#hide').click(function() { $('#text').hide('slow', 'linear') })
      $('#show').click(function() { $('#text').show('slow', 'linear') })
    </script>
  </body>
</html>
```

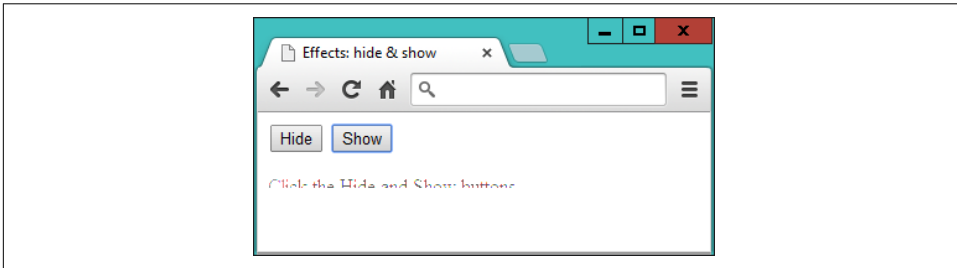


Figure 22-10. The element is in the process of being revealed

The toggle Method

As an alternative to calling both the hide and show methods, you can use the toggle method, which enables you to replace the previous example with **Example 22-12**.

Example 22-12. Using the toggle method

```
<!DOCTYPE html>
<html>
  <head>
    <title>Effects: toggle</title>
    <script src='jquery-3.5.1.min.js'></script>
```

```

</head>
<body>
  <button id='toggle'>Toggle</button>
  <p id='text'>Click the Toggle button</p>
  <script>
    $('#toggle').click(function() { $('#text').toggle('slow', 'linear') })
  </script>
</body>
</html>

```

The toggle method takes the same arguments as hide and show but keeps track of the state of the element internally so that it knows whether to hide or show it.



There are four main jQuery methods that set either one state or another and that offer toggling versions to simplify coding. In addition to toggle, there's fadeToggle, slideToggle, and toggleClass, all described in this chapter.

Fading In and Out

Four methods manage fades: fadeIn, fadeOut, fadeToggle, and fadeTo. You should be getting the idea of how jQuery works by now and realize that the first three are similar to show, hide, and toggle, respectively. The last one, though, is a little different in that it lets you specify an opacity value to which an element (or elements) should be faded, between 0 and 1.

Example 22-13 provides four buttons with which to try out each of these methods, as shown in Figure 22-11.

Example 22-13. The four fade methods

```

<!DOCTYPE html>
<html>
  <head>
    <title>Effects: Fading</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <button id='fadeout'>fadeOut</button>
    <button id='fadeIn'>fadeIn</button>
    <button id='fadetoggle'>fadeToggle</button>
    <button id='fadeto'>fadeTo</button>
    <p id='text'>Click the buttons above</p>
    <script>
      $('#fadeout').click(function() { $('#text').fadeOut( 'slow' ) })
      $('#fadeIn').click(function() { $('#text').fadeIn( 'slow' ) })
      $('#fadetoggle').click(function() { $('#text').fadeToggle( 'slow' ) })
      $('#fadeto').click(function() { $('#text').fadeTo( 'slow', 0.5 ) })
    </script>
  </body>
</html>

```

```

</script>
</body>
</html>

```



Figure 22-11. The text has been faded to 50% opacity

Sliding Elements Up and Down

Another way to make elements disappear and reappear is to alter their height over time to make them appear to slide up and down. There are three jQuery methods to do this: `slideDown`, `slideUp`, and `slideToggle`. They work in a similar way to the previous functions, as coded in [Example 22-14](#) and shown in [Figure 22-12](#).

Example 22-14. Using the slide methods

```

<!DOCTYPE html>
<html>
  <head>
    <title>Effects: Sliding</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <button id='slideUp'>slideUp</button>
    <button id='slideDown'>slideDown</button>
    <button id='slideToggle'>slideToggle</button>
    <div id='para' style='background:#def'>
      <h2>From A Tale of Two Cities - By Charles Dickens</h2>
      <p>It was the best of times, it was the worst of times, it was the age of
        wisdom, it was the age of foolishness, it was the epoch of belief, it was
        the epoch of incredulity, it was the season of Light, it was the season of
        Darkness, it was the spring of hope, it was the winter of despair, we had
        everything before us, we had nothing before us, we were all going direct to
        Heaven, we were all going direct the other way - in short, the period was so
        far like the present period, that some of its noisiest authorities insisted
        on its being received, for good or for evil, in the superlative degree of
        comparison only</p>
    </div>
    <script>
      $('#slideUp').click(function() { $('#para').slideUp( 'slow') })
      $('#slideDown').click(function() { $('#para').slideDown( 'slow') })
      $('#slideToggle').click(function() { $('#para').slideToggle( 'slow') })
    </script>
  </body>
</html>

```

```
</script>
</body>
</html>
```

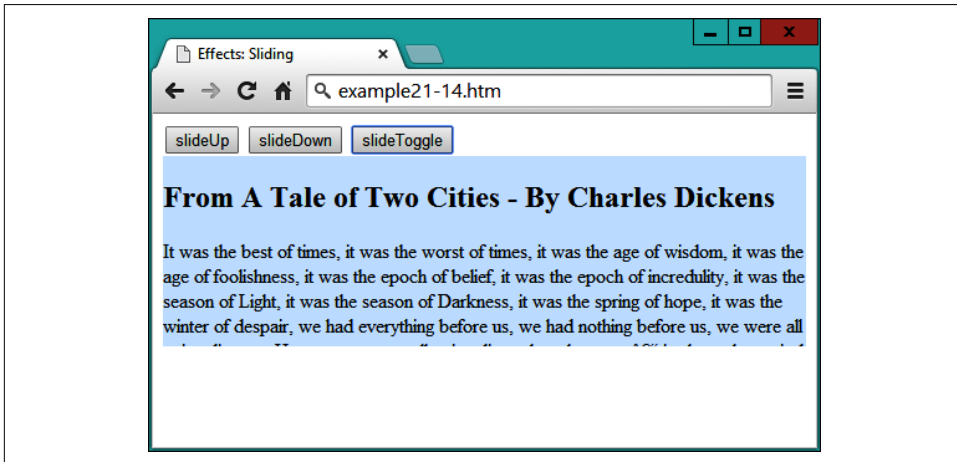


Figure 22-12. The paragraph is sliding up

These methods work well when you have menus and submenus that you want to open up or close dynamically, according to the sections clicked by the user.

Animations

Now we can really start to have some fun by actually moving elements around in the browser. To do this, though, because the default value of `static` will not allow them to move, you must remember to first give your elements' position properties the values of `relative`, `fixed`, or `absolute`.

To animate an element, all you do is supply a list of CSS properties (excluding colors) to the `animate` method. Unlike the previous effects methods, `animate` requires this list of properties first, and then you can supply any duration, easing, and callback arguments you need.

So, for example, to animate a bouncing ball, you could use code such as that in [Example 22-15](#) (which displays like [Figure 22-13](#)).

progress, and `finish` will stop the currently running animation and remove any that are queued.

Let's turn the previous example into a sort of game by making the ball clickable, such that when the click event triggers, the animation will cease. To do this, all that's required is to add the following single line of code underneath the bounce function:

```
$('#ball').click(function() { $(this).finish() })
```

If you successfully manage to click the ball, the `finish` method will stop the current animation, empty the queue, and ignore any callbacks—in other words, the ball will come to rest.

For more information on managing jQuery queues, check out the [documentation for the `queue` method](#), where you'll also learn how to directly manipulate the contents of queues to get exactly the effects you need.

Manipulating the DOM

Because jQuery is so tightly linked with the DOM, of necessity the examples in this chapter have already utilized some of its DOM-accessing methods, such as `html` and `val`. But let's now look at all the DOM methods in detail to discover exactly what you can get access to with jQuery, and how.

In [Example 22-3](#), you saw how to use the `html` method to change the `innerHTML` property of an element. This method can be used either to set the HTML or retrieve it from an HTML document. [Example 22-16](#) (with the jQuery highlighted in bold) shows how to retrieve the HTML content of an element (as shown in [Figure 22-14](#)).

Example 22-16. Displaying the HTML of an element using an alert window

```
<!DOCTYPE html>
<html>
  <head>
    <title>The DOM: html & text</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <h2>Example Document</h2>
    <p id='intro'>This is an example document</p>
    <script>
      alert($('#intro').html())
    </script>
  </body>
</html>
```

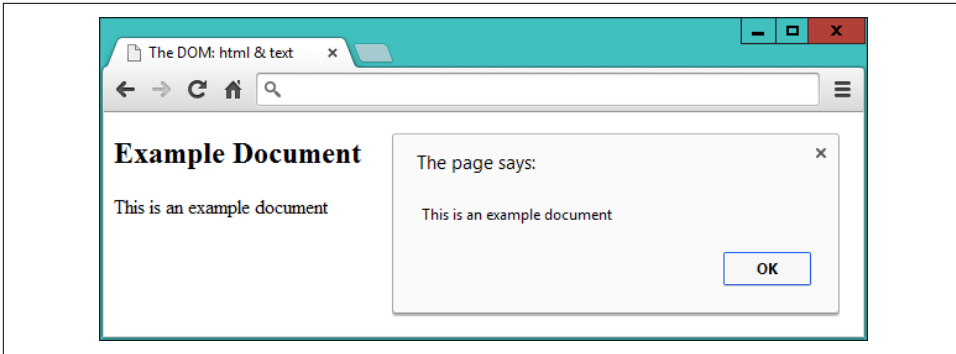


Figure 22-14. Retrieving and displaying the HTML of an element

When you issue this method without arguments, the result is to read rather than set the HTML of the element.

The Difference Between the text and html Methods

When you are working with XML documents, you cannot use the `html` method because it simply won't work (it's designed for use only with HTML). But you can use the `text` method to achieve a similar result (in XML or HTML documents), like this:

```
text = $('#intro').text()
```

The difference between the two is quite simply that `html` treats the content as HTML and `text` treats it as text. So, for example, let's assume you wish to assign the following string to an element:

```
<a href='http://google.com'>Visit Google</a>
```

If you assign it to an HTML element using the `html` method, the DOM will be updated with the new `<a>` element and the link will become clickable. But if you do so to either an XML or HTML document using the `text` method, that string will first be escaped into text (for example, by converting HTML characters such as `<` into the `<` entity, and so on) and then inserted into the element—no element is added to the DOM.

The val and attr Methods

There are a couple more methods for interacting with the content of elements. First, you can set and get the value of an input element with the `val` method, as illustrated in [Example 22-10](#), in which the first and last name fields are read. To set a value, simply provide it as an argument to the method, like this:

```
$('#password').val('mypass123')
```

With the `attr` method, you can get and set the attributes of elements, as shown in [Example 22-17](#), in which a link to the Google website has been completely replaced with one to Yahoo.

Example 22-17. Modifying attributes with the `attr` method

```
<!DOCTYPE html>
<html>
  <head>
    <title>The DOM: attr</title>
    <script src='jquery-3.5.1.min.js'></script>
  </head>
  <body>
    <h2>Example Document</h2>
    <p><a id='link' href='http://google.com' title='Google'>Visit Google</a></p>
    <script>
      $('#link').text('Visit Yahoo!')
      $('#link').attr( { href : 'http://yahoo.com', title : 'Yahoo!' } )
      alert('The new HTML is:\n' + $('#p').html())
    </script>
  </body>
</html>
```

The first jQuery statement uses the `text` method to change the text inside the `<a>` element, and the second one changes the `href` and `title` attribute values to match by supplying the data in the form of an associative array. The third statement displays the changed element's contents in an alert window by first retrieving it with the `html` method, as shown in [Figure 22-15](#).

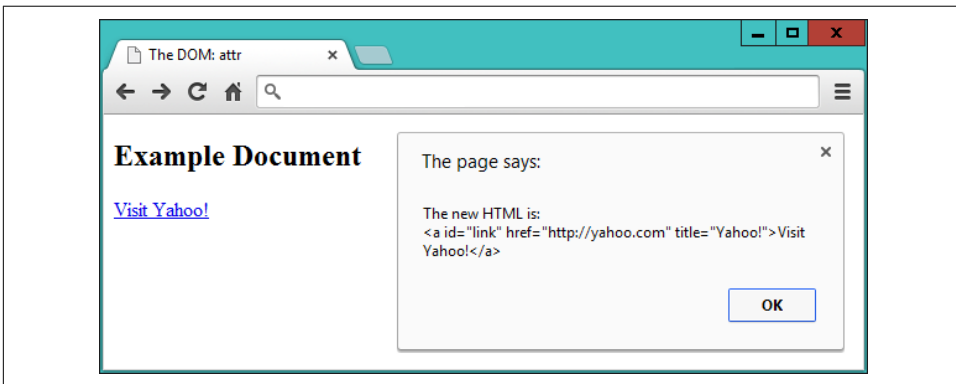


Figure 22-15. The link has now been completely modified

You can also read the value of an attribute like this:

```
url = $('#link').attr('href')
```